

Universidad Gerardo Barrios

Facultad de Ciencia Y Tecnología.

Ingeniería en Sistemas y Redes de Informática.



Universidad
Gerardo Barrios

Desarrollo de aplicaciones con Inteligencia Artificial – Implementación y Escalabilidad

Nombre del proyecto

**Herramienta Autónoma para la Gestión de Ahorros y Presupuestos
Personales**

Nombre de la Aplicación

Ex-Codox

Presentado por:

SMIS013221 - Guevara Jiménez, Emely Alexandra

SMIS083121 - Luna Acosta, Kevin Alexander.

SMIS048021 - Trigueros Blanco, Natalia Alexandra

San Miguel, agosto de 2026

DESCRIPCIÓN DEL SERVICIO Y DEL ENDPOINT CRÍTICO SELECCIONADO

Descripción del servicio

El proyecto Ex-Codox cuenta con un servicio backend denominado Finance AI API, desarrollado con FastAPI, cuyo objetivo es centralizar las operaciones relacionadas con inteligencia artificial y procesamiento de información financiera.

La API obtiene información financiera almacenada en Firebase Firestore, como datos del usuario, billeteras, transacciones, recomendaciones y estadísticas financieras. A partir de esta información construye un perfil financiero que posteriormente puede ser utilizado por diferentes capacidades inteligentes del sistema.

Entre las principales funcionalidades disponibles se encuentran la generación de resúmenes financieros, análisis del comportamiento financiero, recomendaciones personalizadas, predicciones, clasificación del perfil financiero, chatbot financiero y análisis OCR de recibos.

La API también incorpora autenticación mediante Firebase, de forma que los endpoints protegidos solamente pueden ser utilizados por usuarios autenticados. Además, utiliza un mecanismo de caché para reutilizar respuestas generadas previamente cuando la información financiera del usuario no ha cambiado.

Para la Semana 5, este servicio fue instrumentado con mecanismos de observabilidad que permiten registrar el identificador de cada solicitud, la ruta ejecutada, el estado HTTP, la duración del procesamiento y métricas internas relacionadas con las diferentes etapas del servicio.

Endpoint critico seleccionado

Para realizar la medición de rendimiento se seleccionó el siguiente endpoint:

`POST /ai/predict/{uid}`

Este endpoint genera una predicción sobre el comportamiento financiero futuro del usuario utilizando la información financiera almacenada en Firebase y las capacidades de inteligencia artificial integradas en la API.

El endpoint fue seleccionado como flujo crítico porque involucra varias etapas importantes del sistema. Primero se valida la autenticación del usuario y se verifica

que el UID solicitado corresponda al usuario autenticado. Posteriormente, el servicio construye el perfil financiero consultando información almacenada en Firestore, calcula estadísticas, genera una huella digital de los datos y verifica si existe una respuesta válida almacenada en caché.

Cuando existe una respuesta válida en caché, esta puede reutilizarse sin realizar nuevamente una llamada al proveedor de inteligencia artificial. Si no existe una respuesta válida, el sistema construye el contexto financiero, genera el prompt correspondiente, realiza la solicitud al modelo de IA y almacena la respuesta para futuras consultas.

POST `/ai/predict/{uid}` Predecir comportamiento financiero

Genera una predicción del comportamiento financiero futuro del usuario utilizando Inteligencia Artificial.

Parameters [Try it out](#)

Name	Description
uid * required string (path)	UID del usuario registrado en Firebase. uid

Responses

Code	Description	Links
200	Predicción financiera generada mediante IA. Media type application/json Controls Accept header. Example Value Schema <pre>{ "success": true, "message": "Operación ejecutada correctamente.", "data": { "additionalProp1": {} } }</pre>	No links

PREGUNTAS DE OBSERVABILIDAD

Para el endpoint seleccionado se definieron las siguientes preguntas:

¿Qué solicitud se está procesando?	request_id
¿Qué endpoint recibió la solicitud?	method, path
¿La solicitud terminó correctamente?	status_code
¿Cuánto tardó la solicitud completa?	duration_ms
¿Qué operación de IA se ejecutó?	capability
¿La respuesta se obtuvo desde caché?	cache_hit
¿Cuánto tardó la construcción del perfil financiero?	profile_ms
¿Cuánto tardó la consulta de caché?	cache_ms
¿Cuánto tardó la construcción del contexto?	context_ms
¿Cuánto tardó la ejecución de IA?	ai_ms
¿Cuánto tardó el procesamiento interno completo?	total_ms
¿Ocurrió un error durante el procesamiento?	event, level, status_code

Identificación de solicitudes mediante request_id

Cada solicitud recibe un identificador único denominado request_id. Este valor permite relacionar los diferentes eventos generados durante una misma petición.

Por ejemplo, una solicitud puede generar primero un evento interno relacionado con la caché:

```
{ "timestamp": "2020-08-18T00:10:51", "level": "INFO", "logger": "Finance AI",  
  "message": "AI cache hit", "event": "ai_cache_hit", "request_id": "5b52e34b-18  
6b-4c2b-b0e3-341c4e91efe7", "capability": "predict", "cache_hit": true, "profi  
le_ms": 553.89, "fingerprint_ms": 0.22, "cache_ms": 151.71, "context_ms": 0, "  
ai_ms": 0, "save_cache_ms": 0, "save_recommendation_ms": 0, "total_ms": 705.82  
}
```

Posteriormente, el middleware registra la finalización de la misma solicitud:

```
{ "timestamp": "2026-08-16T06:16:51", "level": "INFO", "logger": "finance-ai", "message": "Request completed", "event": "request_completed", "request_id": "5b52e34b-186b-4c2b-b0e3-341c4e91efe7", "method": "POST", "path": "/ai/predict/2s5ajLylt4NlieTDY0SU3a0jWPG3", "status_code": 200, "duration_ms": 720.94 }
```

El mismo request_id permite comprobar que ambos eventos pertenecen a la misma petición. En las pruebas realizadas se registraron correctamente estos eventos junto con las métricas internas y el estado HTTP 200.

CÓDIGO DE INSTRUMENTACIÓN

Para implementar la observabilidad en Finance AI API, se agregó instrumentación que permite registrar cada solicitud mediante un request_id único y medir su tiempo de ejecución.

El middleware registra los principales datos de cada petición:

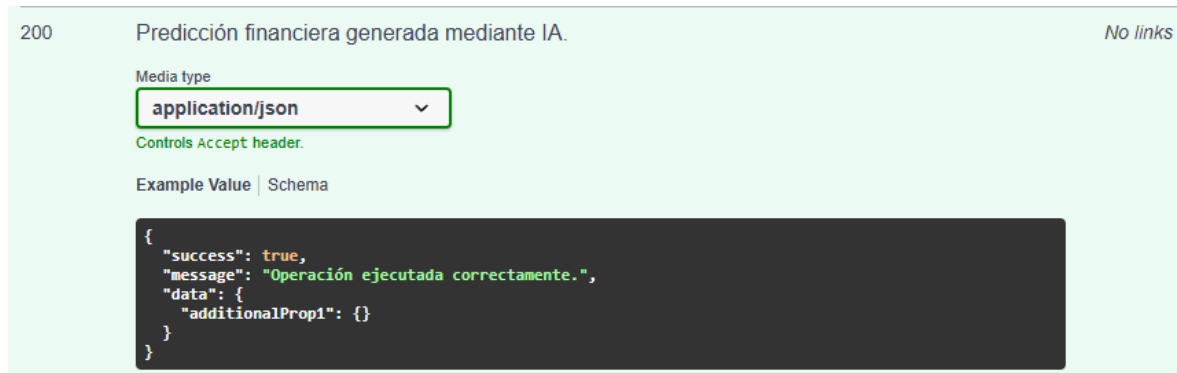
```
logger.info(  
  "AI cache hit",  
  extra={  
    "extra_fields": {  
      "event": "ai_cache_hit",  
      "request_id": request_id,  
      "capability": capability.value,  
      "cache_hit": True,  
      "profile_ms": round(profile_ms, 2),  
      "fingerprint_ms": round(fingerprint_ms, 2),  
      "cache_ms": round(cache_ms, 2),  
      "context_ms": 0,  
      "ai_ms": 0,  
      "save_cache_ms": 0,  
      "save_recommendation_ms": 0,  
      "total_ms": round(total_ms, 2),  
    },  
  },  
)
```

También se instrumentó AIService para registrar métricas internas como profile_ms, cache_ms, ai_ms, total_ms y si la respuesta fue obtenida desde caché.

Evidencia de una solicitud exitosa

Se realizó una solicitud autenticada al endpoint:

POST /ai/predict/{uid}



EVIDENCIA DE UNA ENTRADA INVÁLIDA O ERROR CONTROLADO

Para comprobar el manejo de errores se realizó una solicitud al endpoint:

POST /ai/predict/{uid}

sin proporcionar un Firebase ID Token válido. Debido a que el endpoint requiere autenticación, la API rechazó correctamente la petición con:

HTTP 401 Unauthorized

El sistema registró el evento sin exponer información sensible:

```
"event": "request_completed", "request_id": "cace3c90-a079-4e11-9952-37e2f76f8e46", "method": "POST", "path": "/ai/predict/2s5ajLylt4NlieTDYOSU3a0jWPG3", "status_code": 401, "duration_ms": 107.87}
INFO:      192.168.110.121:61907 - "POST /ai/predict/2s5ajLylt4NlieTDYOSU3a0jWPG3 HTTP/1.1" 401 Unauthorized
```

DATOS EXCLUIDOS POR PRIVACIDAD O SEGURIDAD

Para proteger la información del usuario y las credenciales del sistema, los logs de Finance AI API registran únicamente datos técnicos necesarios para la observabilidad.

Se excluyen datos sensibles como:

- Firebase ID Token y encabezado Authorization.
- API Keys, como la clave utilizada para acceder al servicio de IA.
- Credenciales de Firebase y otras variables secretas.
- Información financiera detallada, como transacciones, saldos o datos personales del usuario.
- Contenido completo de prompts y respuestas generadas por la IA.

En su lugar, los logs almacenan información técnica como request_id, ruta, código de estado, duración, cache_hit y tiempos de procesamiento.

De esta manera se puede analizar el comportamiento y rendimiento de la API sin exponer credenciales ni información financiera sensible de los usuarios.

ESCENARIO Y RESULTADOS DE LA MEDICIÓN DE RENDIMIENTO

Para evaluar el rendimiento de Finance AI API, se realizó una prueba sobre el endpoint crítico:

POST /ai/predict/{uid}

Elemento	Configuración
Ambiente	API ejecutada localmente mediante Docker
Endpoint	POST /ai/predict/{uid}
Autenticación	Firebase ID Token
Herramienta	benchmark_baseline.py
Solicitudes	20 solicitudes secuenciales

Resultado esperado	HTTP 200 OK
--------------------	-------------

El script midió el tiempo de respuesta de cada solicitud y calculó p50, p95, tiempo máximo y tasa de error.

Resultados de la línea base

Métrica	Resultado
Solicitudes	20
Exitosas	20
Fallidas	0
p50	601.40 ms
p95	1003.15 ms
Máximo	1165.79 ms
Tasa de error	0.00%

```
=====
RESULTADOS DE LA LÍNEA BASE
=====
Solicitudes totales : 20
Solicitudes exitosas: 20
Solicitudes fallidas: 0
Tasa de error      : 0.00%

p50                : 601.40 ms
p95                : 1003.15 ms
Máximo             : 1165.79 ms
=====

TABLA DE LÍNEA BASE

| Métrica | Resultado |
|---|---:|
| Solicitudes | 20 |
| Exitosas | 20 |
| Fallidas | 0 |
| Tasa de error | 0.00% |
| p50 | 601.40 ms |
| p95 | 1003.15 ms |
| Máximo | 1165.79 ms |
```


CUELLO DE BOTELLA, RIESGO O COMPORTAMIENTO INESPERADO IDENTIFICADO

Mediante los logs y las métricas por etapas se identificó que el principal cuello de botella se encuentra en la construcción del perfil financiero mediante:

```
build_finance_profile()
```

Este proceso realiza varias consultas a Firestore para obtener información del usuario, billeteras, transacciones, recomendaciones y otros datos necesarios para construir el contexto financiero.

Durante las pruebas se observaron solicitudes donde:

```
profile_ms = 1833.49 ms
```

```
cache_ms  = 184.02 ms
```

```
ai_ms     = 0 ms
```

```
total_ms  = 2017.62 ms
```

MEJORA APLICADA O PROPUESTA

A partir del cuello de botella identificado, se aplicaron optimizaciones para reducir las consultas y el procesamiento innecesario de datos.

Las principales mejoras realizadas fueron:

- Se limitó la consulta de recomendaciones en Firestore a las 5 más recientes, evitando recuperar y ordenar todo el historial.
- Se optimizó StatisticsService para calcular el mayor ingreso, mayor gasto y categoría favorita en un solo recorrido de las transacciones, en lugar de recorrerlas varias veces.
- Se reorganizó la construcción del perfil financiero para evitar operaciones redundantes.

Con estas mejoras se busca disminuir la cantidad de datos recuperados desde Firestore y reducir el procesamiento realizado en cada solicitud.

Como mejora futura, se propone precalcular y almacenar un resumen financiero del usuario, evitando reconstruir el perfil completo y procesar todas las transacciones cada vez que se solicita una predicción.

Estas optimizaciones permiten mejorar el rendimiento actual y preparar la API para soportar un mayor crecimiento en usuarios y datos.

PLAN DE ESCALABILIDAD Y LIMITACIONES PENDIENTES

Plan de escalabilidad

A partir de las mediciones realizadas, el plan de escalabilidad se enfoca primero en reducir el costo de las consultas a Firestore y la reconstrucción del perfil financiero, antes de aumentar la cantidad de instancias o workers.

Las principales acciones propuestas son:

- Precalcular el resumen financiero del usuario para evitar procesar todas las transacciones en cada solicitud.
- Utilizar caché para reutilizar resultados cuando la información financiera no haya cambiado.
- Limitar y paginar las consultas a Firestore cuando aumente la cantidad de transacciones.
- Utilizar colas o tareas asíncronas para procesos que no requieran una respuesta inmediata.

Los principales indicadores para decidir cuándo escalar serán el p95, la tasa de error, el número de solicitudes concurrentes y el consumo de CPU y memoria.

Limitaciones pendientes

Actualmente se identifican las siguientes limitaciones:

- El perfil financiero todavía requiere varias consultas a Firestore.
- Las transacciones del usuario son recuperadas para calcular estadísticas, lo que puede aumentar el tiempo de procesamiento conforme crezca el historial.
- La latencia de Firebase y del proveedor de IA son factores externos que pueden variar.

- La caché actual evita llamadas innecesarias a la IA, pero todavía requiere construir el perfil financiero para comprobar su validez.
- No se ha implementado todavía infraestructura de autoescalado, balanceador de carga ni un sistema de colas.